

AI 辅助漏洞挖掘

系统设计指南

从零开始，设计你自己的 AI 安全测试平台

小东

版本 1.0 | 2026 年 06 月

目录

- 第一章 概述：什么是 AI 辅助漏洞挖掘
- 第二章 核心设计哲学：不束缚 AI，只设边界
- 第三章 系统总体架构
- 第四章 核心技能文件设计（灵魂章节）
- 第五章 初始提示词工程
- 第六章 会话调度与并发控制
- 第七章 会话生命周期管理
- 第八章 终态判定与证据系统
- 第九章 实时通信与前端展示
- 第十章 漏洞报告自动化
- 第十一章 安全防护机制
- 第十二章 从零搭建：设计你自己的系统

第一章 概述：什么是 AI 辅助漏洞挖掘

1.1 背景与动机

传统渗透测试依赖安全工程师手工操作：抓包、逐个测试参数、查阅知识库、编写验证脚本。一个经验丰富的安全研究员同时测试的目标通常不超过两三个。

大语言模型改变了这一局面。现代顶级 AI 模型本身就有丰富的安全知识——它知道什么是 SQL 注入，知道怎么测试 IDOR，知道 SSRF 的各种变种。它不需要你教它怎么做渗透测试。

那我们还需要做什么？答案是：给它一个运行平台，告诉它边界在哪。

1.2 核心思路：不教方法，只设边界

整个系统的设计哲学用一句话概括：

「AI 已经会做渗透测试，我们只需要告诉它什么不该做、什么不该报。」

这传统自动化扫描完全不同。传统工具需要你定义每条规则、每个 payload、每种判断逻辑。AI 不需要——它有自己的知识和推理能力。我们要做的：

- 设定行为边界——什么操作是危险的、不允许的
- 设定报告标准——什么才算真正的漏洞，什么只是现象
- 提供基础设施——运行环境、调度平台、报告流程
- 然后放手——不干涉测试思路、不限制攻击路径、不规定测试顺序

1.3 灵魂金句

「现象不是漏洞，漏洞是结果。报的是结果（越权/注入/RCE），不是过程（信息泄露/配置问题）。」

这句话定义了整个系统的报告哲学。AI 在测试中会发现大量的「现象」——源码泄露、跨域配置、安全头缺失、版本号暴露——但这些都不是漏洞。漏洞是这些现象被利用后造成的实际后果：数据被窃取、权限被绕过、命令被执行。

1.4 与传统方案的区别

维度	传统自动化扫描	AI 辅助漏洞挖掘
知识来源	人类编写的规则和 payload	AI 模型自带的安全知识
测试逻辑	预定义流程，线性执行	AI 自主推理，动态决策
适应性	遇到新情况不会了	根据返回结果灵活调整
人类角色	定义所有规则	只定义边界
核心文件	几十上百个规则文件	一个核心技能文件

第二章 核心设计哲学：不束缚 AI，只设边界

2.1 为什么不需要几十个技能文件

很多人的第一反应是：给每种漏洞写一个详细的测试方法论。SQL 注入一个、XSS 一个、SSRF 一个……最后搞出几十个文件。

这是错的：

- AI 已经知道怎么测试了——训练数据包含了海量安全知识，它知道的可能比你写的还多
- 过度指导限制 AI 思路——你告诉它「先 A 再 B 再 C」，它就丧失了自主推理的能力
- 维护成本极高——几十个文件，上万行，改一个技术要动好几个文件
- 上下文浪费——每加载一个文件就占用宝贵的上下文窗口

2.2 一个核心技能文件就够了

整个系统只需要一个核心技能文件。它告诉 AI 三件事：

- 什么不该做（行为边界）
- 什么不该报（报告过滤器）——这是最重要的部分
- 报告要达到什么标准（质量门槛）

其余的——测什么漏洞、用什么 payload、走什么路径——全部交给 AI 自己决定。

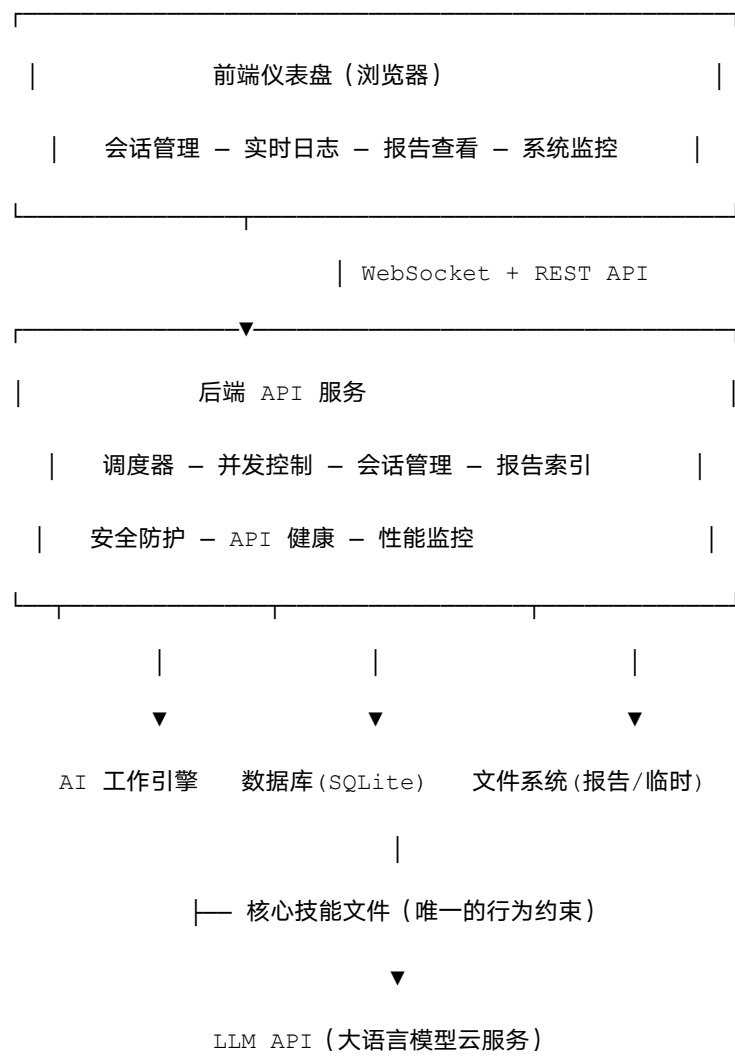
2.3 信任 AI 的判断力

- 信任 AI 的测试方法选择——它会根据目标自己决定先测什么
- 信任 AI 的推理能力——它能判断有没有越权可能
- 信任 AI 的知识储备——它知道各种漏洞类型和绕过技巧
- 不信任 AI 的自制力——所以要限制危险操作
- 不信任 AI 的报告标准——所以要明确什么值得报

核心原则：解放 AI 的思考能力，约束 AI 的行为边界。

第三章 系统总体架构

3.1 架构图



3.2 组件说明

组件	职责	关键特性
----	----	------

前端仪表盘	可视化操作界面	会话管理/实时日志/报告查看/系统监控
调度器	管理会话创建/排队/执行/停止	优先级排序，按项目公平调度
并发控制器	控制同时运行的 AI 会话数	优先级队列，动态调整容量
会话工作器	与 AI 交互，执行测试	自动重连/超时/临时文件监控
状态检测器	判断会话最终结果	12 条决策规则，基于证据判定
报告索引器	扫描/解析/存储报告	等级过滤/邮件通知
事件总线	内部消息分发	发布/订阅，支持多 WebSocket 客户端
安全防护	防止 AI 失控	磁盘监控/自循环检测/API 健康探测

AI 工作引擎的关键设计：

- 每个会话一个独立 AI 工作进程，互不干扰
- AI 有命令行权限（curl、脚本、浏览器自动化）
- 环境变量隔离每个会话的临时目录
- 启动时只加载一个核心技能文件，然后放手让 AI 自主测试

第四章 核心技能文件设计（灵魂章节）

这是整个系统最重要的一章。核心技能文件是你的系统和别人系统之间的全部差距。它只有几百行，但每一行都是你的漏洞挖掘经验和判断力的结晶。

4.1 六大设计准则

核心技能文件的设计遵循以下六大准则。这不是建议，而是铁律——违反任何一条都会导致系统输出大量垃圾。

准则一：铁律置顶，垃圾洞清单开篇

核心技能文件的第一部分必须是「绝对不报的垃圾洞清单」。不是放在中间，不是放在最后，而是放在最开头——AI 第一眼看到的就是这个。

为什么置顶？因为 AI 在长对话中会逐渐遗忘早期指令。放在开头意味着即使后面的内容被遗忘了，这个清单还在记忆中。

垃圾洞清单应该包括但不限于：

绝对不报	原因	价值
CORS 跨域配置	跨域本身不是漏洞，除非能证明窃取了具体数据	0 元
Sourcemap 泄露	配置问题，不是可利用的安全漏洞	0 元
HTTP 安全头缺失	X-Frame-Options/CSP 等缺失是理论风险	0 元

版本号/中间件指纹	信息收集的副产品，不是漏洞	0 元
Self-XSS	只能攻击自己的 XSS，没有实际危害	0 元
SSL/TLS 配置警告	除非是严重降级攻击	0 元
单独的开放重定向	无法链式利用的重定向几乎无害	0 元
Rate limiting 缺失	功能建议，不是漏洞	0 元
任何没有 PoC 的发现	不能重现的就不存在	0 元

准则二：灵魂金句——现象不是漏洞，漏洞是结果

「现象不是漏洞，漏洞是结果。报的是结果（越权/注入/RCE），不是过程（信息泄露/配置问题）。」

这句话必须写在核心技能文件的显眼位置，作为 AI 报告时的最高判断标准。

具体来说：

这是现象（不报）	这是结果（报）
发现了一个内部 API 路径	通过该路径实现了越权访问其他用户数据
JS 里暴露了一个 API Key	用该 Key 成功调用了付费接口
CORS 允许任意来源	通过 CORS 配合 XSS 窃取了用户 token
发现了 debug 接口	通过 debug 接口执行了任意命令

源码中有 SQL 拼接	构造了 payload 成功提取了数据库内容
-------------	------------------------

AI 在写报告之前必须自问：「我报的是一个现象还是一个结果？」如果答案是现象，不报。

准则三：指挥官只装决策逻辑，深度 payload 下沉到模型本身

核心技能文件里不要写具体的 payload、不要写详细的注入语法、不要写 WAF 绕过的编码方式。这些东西 AI 全都知道。

核心技能文件应该只包含：

- 决策树——按目标特征（登录态/技术栈/功能类型）动态分支的决策逻辑
- ROI 目标选择——哪些功能点最值得测试（如支付/认证/数据导出）
- Phase 切换检查点——什么时候该换方向（如「20 分钟无进展 → 切换攻击面」）

不应该包含：

- SQL 注入的 payload 列表（AI 知道）
- XSS 的绕过技巧（AI 知道）
- SSRF 的内网地址段（AI 知道）
- 任何「怎么做」的详细方法论（AI 知道）

原则：你不需要教鲨鱼怎么游泳。你只需要告诉它哪片海域有鱼，哪片有暗礁。

准则四：防遗忘机制——速查卡与 Phase 切换检查点

AI 在长对话中会逐渐遗忘早期指令。这是所有大语言模型的固有缺陷。核心技能文件必须包含对抗遗忘的机制。

速查卡

在核心技能文件中设计一个简洁的「速查卡」区域——用最精炼的语言把核心规则浓缩成 10-20 条短句。例如：

- CORS ≠ 漏洞
- 无 PoC ≠ 漏洞
- 现象 ≠ 结果
- Self-XSS = 垃圾
- 20 分钟无进展 → 换方向
- 报告必须有 curl
- P3 以下不写报告

Phase 切换检查点

在核心技能文件中要求 AI 每次切换测试阶段时，必须回头重读一遍速查卡。这样即使在对话的第 100 轮，AI 仍然记得核心规则。

实现方式：在技能文件中写明——

- 「每次你决定切换到新的测试方向之前，必须先回顾上方的速查卡，确认你没有遗忘任何规则。」
- 「如果你已经连续测试超过 30 分钟，暂停并重读速查卡。」

准则五：七问验证门——报告前的终极过滤

在核心技能文件中设定：AI 在写报告之前，必须逐条自答以下 7 个问题。只有全部通过才能写报告。

#	问题	不通过则
1	这个发现在授权范围内吗？	停止，不报
2	我有完整的 PoC 吗（可重现的 curl/HTTP 请求）？	补充 PoC，没有就不报
3	我需要假设或推测来解释它的危害吗？	如果需要假设，说明危害不够直接，不报
4	影响是已证明的还是「可能」的？	「可能」不报，只报已证明的
5	这是一个现象还是一个结果？	现象不报
6	一个完全不懂安全的开发者看到这个报告，能理解危害吗？	写得更清楚，或者危害不够明显不报
7	这个报告发到漏洞平台，会被接受还是会被关闭？	会被关闭 → 不报

第 7 问是终极检验。如果你自己都觉得漏洞平台会关掉这个报告，那就不要浪费时间写。

准则六：决策树而非固定流程

绝对不要在核心技能文件里写「第一步做什么、第二步做什么」的固定流程。取而代之的是决策树——根据目标的实际情况动态选择测试方向。

决策树的设计思路：

- 按登录态分支——有登录态 → 优先测越权/IDOR；无登录态 → 优先测未授权访问/注入
- 按技术栈分支——Java 中间件 → 关注反序列化/JNDI；PHP → 关注文件包含/SQL 注入
- 按功能分支——支付功能 → 测竞态/金额篡改；上传功能 → 测文件上传/路径穿越
- 按 ROI 分支——高价值功能（认证/支付/数据导出）优先测试

关键的时间约束：

- 20 分钟无进展 → 必须换方向，不要死磕
- 这条规则写进核心技能文件，防止 AI 在一个死角浪费大量时间

决策树不是穷举所有情况，而是给 AI 一个「怎么选择下一步」的框架。具体怎么做，AI 自己决定。

4.2 文件结构总览

根据六大准则，核心技能文件的结构如下：

顺序	区域	内容	长度建议
1	垃圾洞清单（置顶）	绝对不报的东西列表	20-30 行
2	灵魂金句	「现象不是漏洞，	3-5 行

		漏洞是结果」	
3	速查卡	核心规则浓缩版 (10-20 条短句)	15-20 行
4	铁律	不能做的危险操作	10-15 行
5	七问验证门	报告前必须通过的 7 个问题	15-20 行
6	决策树	按登录态/技术栈/功 能动态分支	30-50 行
7	报告格式要求	格式/等级/PoC 要 求	10-15 行
8	终止协议	状态标记规范	5-10 行
9	防遗忘指令	Phase 切换时重读 速查卡	5-10 行

总长度：150-200 行。绝对不要超过 300 行。

4.3 迭代方式

核心技能文件的迭代非常简单：

- AI 报了噪音 → 加到垃圾洞清单
- AI 做了危险操作 → 加到铁律
- AI 报告质量差 → 强化七问验证门或报告标准
- AI 在某个方向死磕太久 → 调整决策树的时间约束

- 新的噪音模式出现 → 加到速查卡

每一次实战都是迭代核心技能文件的机会。这个文件是你的经验沉淀。

4.4 常见误区

误区	为什么错	正确做法
给每种漏洞写技能文件	AI 已经会了，你在做无用功	一个文件，只写边界
写几千行方法论	浪费上下文，AI 反而更差	150-200 行足够
规定固定测试流程	限制 AI 灵活性	用决策树动态分支
不写垃圾洞清单	AI 报大量噪音	置顶，最重要的部分
没有防遗忘机制	AI 在长对话中遗忘规则	速查卡 + Phase 检查点
没有七问验证门	AI 报「可能」的漏洞	7 问全过才能报

第五章 初始提示词工程

5.1 提示词组成

部分	内容	目的
授权文档	测试授权证明	AI 知道测试合法
核心技能	行为边界+报告标准	约束行为和输出
报告路径	绝对路径	AI 知道报告写哪里
临时目录	隔离路径	每个会话独立
目标信息	URL + 已知上下文	AI 知道测什么

5.2 设计原则

- 授权前置——第一时间看到，打消顾虑
- 边界清晰——核心技能内容放显眼位置
- 路径明确——全用绝对路径
- 不要废话——越简洁，留给测试的上下文越多

5.3 快速指令（可选）

- 「深入调查」——对某个发现深入测试
- 「生成报告」——立即生成当前发现的报告
- 「换个方向」——尝试其他测试角度

第六章 会话调度与并发控制

6.1 优先级队列

- 每个会话有优先级值（数字越大越优先）
- 空闲槽位时最高优先级先执行
- 支持动态调整，同优先级按时间排序

实现用最小堆，插入取出都是 $O(\log n)$ 。

6.2 并发容量控制

维度	说明	建议值
全局上限	所有项目共享	5-10
每项目上限	单项目	全局的 1/2
软限制	弹性上限	全局 + 1-2
硬限制	绝对上限	全局 + 2-3

- 异步上下文管理器确保槽位必定释放
- 运行时动态调整容量

6.3 公平调度与批量操作

- 释放槽位时按项目轮询，防止大项目饿死小项目
- 批量操作分块处理（每次 200），块间让出控制权

第七章 会话生命周期管理

7.1 状态机

[创建] → queued（排队）

queued → running（开始测试）

running → vuln_found / low_roi / need_input / error / stopped

终态	含义	后续
vuln_found	发现有 PoC 的真实漏洞	查看报告，提交
low_roi	无有价值发现	换方法或跳过
need_input	需要人工输入	提供后恢复
error	系统错误	排查重试
stopped	用户停止	按需重启

7.2 工作器与流式处理

- 每个会话一个独立异步协程
- 接收流式输出，分发到事件总线
- 监控临时目录大小，检测 API 健康
- 超时用「无活动超时」而非「总时长超时」
- 状态标记只从最后一条消息的独立行提取

7.3 自动重连

- 最多 3 次，指数退避间隔
- 恢复上下文，从中断处继续
- API 普遍不健康时暂停所有新会话

第八章 终态判定与证据系统

8.1 双重验证

不单纯相信 AI。同时检查：

- AI 的状态标记声明（它说了什么）
- 磁盘上的物理证据（它实际做了什么）

8.2 有效报告标准

- 严重等级 P1/P2/P3
- 有标题
- 正文 >= 200 字符
- 包含复现证据（curl/HTTP 请求/URL）

8.3 十二条决策规则

#	条件	结果
1	被用户停止	stopped
2	临时目录超限	error
3	超硬性时间上限	error
4	VULN_FOUND + 有效报告	vuln_found
5	VULN_FOUND + 无报告	low_roi（降级）

6	LOW_ROI/NEED_INPUT + 有报告	vuln_found (升级)
7	LOW_ROI + 无报告	low_roi
8	NEED_INPUT + 无报告	need_input
9	无标记 + 有报告	vuln_found (补救)
10	无标记 + 正常结束	error (协议违规)
11	无标记 + 异常结束	error
12	兜底	error

- 物理证据可以推翻 AI 声明 (规则 5/6)
- 有证据就不浪费 (规则 9)
- 没证据就不认 (规则 5)

第九章 实时通信与前端展示

9.1 事件总线

- 发布/订阅模式，每个会话独立通道
- 每个订阅者独立队列（上限 1024），满则丢弃

9.2 WebSocket

端点	用途
会话流	单个会话的实时 AI 输出
仪表盘流	所有会话的状态概览

- 前端也可通过 WebSocket 发控制命令

9.3 前端页面

- 仪表盘——状态汇总 + 最近漏洞 + 系统资源
- 会话列表——筛选/排序/批量操作
- 会话详情——实时日志 + 漏洞列表
- 报告页——按等级排序，Markdown 渲染，导出
- 设置——并发/模型/超时/通知/授权文件

第十章 漏洞报告自动化

10.1 报告格式

severity: P1

title: 漏洞标题

target: 目标

type: 类型

date: 日期

漏洞描述

影响范围

复现步骤（含 curl）

PoC

修复建议

10.2 索引与通知

- 扫描报告目录，解析元数据，只索引 P1/P2/P3
- 存入数据库，去重检测

- SMTP 邮件通知，每个高危漏洞立即发送

第十一章 安全防护机制

11.1 磁盘防护

- 第一层：自循环检测——写入路径在读取路径子目录内 → 拦截
- 第二层：物理限制——每个会话临时目录上限 5GB，每 3 秒检查

11.2 API 健康监控

- 连续失败 → 判定不健康 → 指数退避探测（10/20/40/80/160/240 分钟）
- 探测成功自动恢复，步数 ≥ 3 邮件通知

11.3 性能监控

指标	警告	危险	处理
CPU	70%	90%	减少并发
内存	75%	90%	停止新会话
磁盘	80%	95%	全面停止

- 渐进降级，自动恢复（有滞后防振荡）

11.4 孤儿进程与启动恢复

- 每 5 分钟清理孤儿 AI 子进程
- 启动时：running → queued，重扫报告索引，清理临时文件

第十二章 从零搭建：设计你自己的系统

12.1 阶段一：最小版本（1-2 周）

目标：一个会话、一个目标、命令行

- 1. 选一个支持工具调用的 LLM API
- 2. 写核心技能文件（垃圾洞清单 + 铁律 + 七问 + 报告标准 + 终止协议）
- 3. 写脚本：构建提示词 → 调用 AI → 保存输出
- 4. 手动运行，观察 AI 行为
- 5. 根据表现迭代核心技能文件——这是最重要的步骤

这个阶段的产出不是系统，而是你对 AI 行为的理解。它报了什么不该报的？

这些就是你往垃圾洞清单里加的条目。

12.2 阶段二：平台化（2-4 周）

- 1. 搭建后端 API + 会话管理 + 并发控制
- 2. 终态判定（标记 + 报告验证）
- 3. 简单前端 + WebSocket 实时推送

组件	推荐	理由
后端	FastAPI	异步原生
前端	React / Vue	生态成熟
数据库	SQLite	零运维
通信	WebSocket	低延迟

12.3 阶段三：加固（4-8 周）

- 磁盘防护 + API 健康监控 + 性能监控
- 优先级调度 + 自动重连 + 批量操作
- 报告索引 + 邮件通知 + 完善前端

12.4 持续迭代

系统上线后唯一重要的事：迭代核心技能文件。

- 每次实战后分析：AI 报了什么噪音？做了什么不该做的？
- 更新垃圾洞清单、铁律、速查卡
- 这个文件就是你的核心竞争力

12.5 设计原则总结

原则	说明
边界而非方法论	告诉 AI 什么不该做，不教它怎么做
一个文件够了	150-200 行，简洁有力
信任 AI 的能力	它知道怎么测试，你只管边界
垃圾洞清单置顶	AI 第一眼看到的的就是不该报什么
七问验证门	每个报告都必须通过终极过滤
决策树不是流程	动态分支，20 分钟无进展换方向
防遗忘	速查卡 + Phase 检查点

证据优先	物理证据 > AI 声明
持续迭代	每次实战都是优化机会

— 全文完 —

祝你构建出自己的 AI 安全测试系统！